How to read the list

In [2]:
```
list1=[1,2,3,4]
list1
```

Out[2]:
```
[1, 2, 3, 4]
```

In [3]:
```
list2=['A','B','C']
list2
```

Out[3]:
```
['A', 'B', 'C']
```

In [4]:
```
list3=['A','B',10,20]
list3
```

Out[4]:
```
['A', 'B', 10, 20]
```

In [5]:
```
list4=['A',10,10.4,10+20j,0b111,True]
list4
```

Out[5]:
```
['A', 10, 10.4, (10+20j), 7, True]
```

In [6]:
```
list5=[10,10,10]
list5
```

Out[6]:
```
[10, 10, 10]
```

In [7]:
```
list6=[[1,2,3]]
list6
```

Out[7]:
```
[[1, 2, 3]]
```

- List has array of elements in square bracket
- List elements has any type of data
- That means list follow heterogeneous concept
- List allows different data types
- List allows duplicates
- List allows list in list

In [10]:
```
a=10
b=20
c=30
[a,b,c]
```

Out[10]:
```
[10, 20, 30]
```

type

In [11]:
```
type(list1)
```

Out[11]:
```
list
```

len

In [13]:
```
len(list1)
```

Out[13]:
```
4
```

In [14]:
```
list2=[1,2,3,4,5,'A','B','C','D']
len(list2)
```

Out[14]:
```
9
```

max-min-sum

In [15]:
```
list1=[1,2,3,4]
# apply max keyword on above list1
# apply min keyword on above list1
# apply sum keyword on above list1
print(max(list1))
print(min(list1))
print(sum(list1))
```

Out[15]:
```
(4, 1, 10)
```

In [16]:
```
list2=['A','B','C','D']
# apply max keyword on above list2 D
# apply min keyword on above list2 A
# apply sum keyword on above list2
print(max(list2))
print(min(list2))
print(sum(list2))
```

```
D
A
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[16], line 7
      5 print(max(list2))
      6 print(min(list2))
----> 7 print(sum(list2))

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [19]:
```
list3=[1,2,3,4,'A','B','C','D']

# max  min  sum  cant apply for heterogeneous list
# max and min can apply homogeneous list
# sum cant apply for strings
```

```
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[19], line 2
      1 list3=[1,2,3,4,'A','B','C','D']
----> 2 sum(list3)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- type
- max
- min
- sum

in

In [24]:
```
list1=[1,2,3,4]

1 in list1
2 in list1
3 in list1
4 in list1
5 in list1

i in list1
```

Out[24]:
```
False
```

In [25]:
```
for i in list1:
    print(i)
```

```
1
2
3
4
```

In [26]:
```
# WAP add the elements in a given list
list1=[1,2,3,4]
summ=0
for i in list1:
    summ=summ+i

print(summ)
```

```
10
```

In [27]:
```
sum(list1)
```

Out[27]:
```
10
```

index

In [30]:
```
#1,2,'A','B'
#0 1  2   3
list1=[1,2,'A','B']
list1[0]
list1[1]
list1[2]
list1[3]

list1[i]
```

Out[30]:
```
'B'
```

In [33]:
```
for i in range(len(list1)):
    print(list1[i])
```

```
1
2
A
B
```

In [40]:
```
list1=[1,2,3,'','a','b']
len(list1)
```

Out[40]:
```
6
```

In [ ]:
```
LIST
Dictionary
lambda functions
file handling sessions

==============tuple write up work===========
27th saturday : LLM API chatgpt  gemini

statistics theory part : 8-10 sessions

3rd feb github: resume
EDA with python

Python OOPS ===== ML using python
```

- how to read list
- type
- max
- min
- sum
- different types of list
- in
- index

In [1]:
```
list1=[1,2,3]
list1[0]
```

Out[1]: 1

mutable-immutable

In [3]:
```
# I want to change 1 value into 100
# by using index
list1[0]=100
```

In [4]:
```
list1
```

Out[4]: [100, 2, 3]

In [5]:
```
list1=[1,2,3]
list1[0]=100
list1
```

Out[5]: [100, 2, 3]

Note: Lists are mutable

In [7]:
```
str1='python'
str1[0]='P'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call las
t)
Cell In[7], line 2
      1 str1='python'
----> 2 str1[0]='P'

TypeError: 'str' object does not support item assignment
```

In [15]:
```
list1=[['A','B']]
# retrive the index of 'A'
v1=list1[0]
v1[0]

list1[0][0], list1[0][1]
```

Out[15]: ('A', 'B')

In [18]:
```
list2=['A',[1,2]]
# Retrive 2
list2[1][1]

len(list2) # two elements
```

Out[18]: 2

In [35]:
```
list3=[[[[[[[['Apple']]]]]]]]
len(list3)
# len of list3 says 1
# only 1 element is there
# what is the index 0
list3[0][0][0][0][0][0][0][0]
```

Out[35]: 'Apple'

In [45]:
```
list3=[[[[[['Cherry',[['Apple']]]]]]]]
list3[0][0][0][0][0][0] # 'Cherry'
list3[0][0][0][0][0][1][0][0] # 'Apple'
```

Out[45]: 'Apple'

In [47]:
```
list3[0][0][0][0][0][1][0][0].upper()
```

Out[47]: 'APPLE'

In [52]:
```
list1=['hyd','bengaluru','chennai']
# print the elements having len >3
# bengaluru and chennai

# iterate over the list using for loop
# each element will come
# if condition
#    print that element
###################### i means directly element
for i in list1:
    if len(i)>3:
        print(i)

################### i means index
for i in range(len(list1)):
    if len(list1[i])>3:
        print(list1[i])
```

```
bengaluru
chennai
bengaluru
chennai
```

In [55]:
```
list1=['hyd','bengaluru','chennai']
# print the count having len >3
count=0
for i in list1:
    if len(i)>3:
        count=count+1
print(count)
```

```
2
```

In [58]:
```
list1=['bengaluru','hyd','chennai']
# sum the index of elements having len >3
# bengaluru index=0
# chennai index=2
# 0+2=2

# you want indexes ===== in or range
summ=0
for i in range(len(list1)):
    if len(list1[i])>3:
        print(i,list1[i])
        summ=summ+i
print(summ)
```

```
0 bengaluru
2 chennai
2
```

In [59]:
```
list2=['beng#aluru','hyd','chen#nai']
# print the element which are having '#'
# if '#' in <element>
for i in list2:
    if '#' in i:
        print(i)

# how to create a new list
# to save the answer
# instead of print
```

```
beng#aluru
chen#nai
```

list-methods

In [62]:
```
dir([])
```

Out[62]:
```
['__add__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'clear',
 'copy',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']
```

In [ ]:
```
'append',
'clear',
'copy',
'count',
'extend',
'index',
'insert',
'pop',
'remove',
'reverse',
'sort'
```

append

In [ ]:
```
# you want to create a list with some elements
# then use append method
```

In [67]:
```
#Append object to the end of the list
list1=[]
list1.append(100)
list1.append(200)
list1.append('apple')
list1
```

Out[67]:
```
[100, 200, 'apple']
```

In [68]:
```
list2=[1,2,3]
list2.append(1000)
list2
```

Out[68]:
```
[1, 2, 3, 1000]
```

In [69]:
```
list1=[100,200]
list1.append(['Apple','Banana'])
list1
```

Out[69]:
```
[100, 200, ['Apple', 'Banana']]
```

In [72]:
```
# Create a list of numbers from 10 to 20
list1=[]
for i in range(10,21):
    list1.append(i)

list1
```

Out[72]:
```
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

In [75]:
```
list1=['hyd','bengaluru','chennai']
# save the elements in a list having len >3
# ['bengaluru','chennai']
out_put=[]
for i in list1:
    if len(i)>3:
        out_put.append(i)
out_put
```

Out[75]:
```
['bengaluru', 'chennai']
```

In [79]:
```
list1=['hyd','bengaluru','chennai']
# save the elements in a list other than hyd
# ['bengaluru','chennai']
# step-1: take empty list
# step-2: iterate over given list
# step-3: apply the condition
# step-4: append the output values in an empty list

out=[]
for i in list1:
    if i!='hyd':
        out.append(i)
out
```

Out[79]:
```
['bengaluru', 'chennai']
```

In [80]:
```
out=[]
for i in list1:
    if i=='hyd':
        out.append(i)
out
```

Out[80]:
```
['hyd']
```

In [82]:
```
list2=['beng#aluru','hyd','chen#nai']
# save the element which are having '#'
# if '#' in <element>
out=[]
for i in list2:
    if '#' in i:
        out.append(i)

out
```

Out[82]:
```
['beng#aluru', 'chen#nai']
```

In [86]:
```
# create a list of 10 elements,
# having a random numbers between 10 to 100
import random
num=[]
for i in range(10):
    num.append(random.randint(10,100))

num
```

Out[86]: [92, 80, 35, 14, 77, 25, 24, 92, 85, 95]

In [ ]:
```
#########################################################
list1=[]
for i in range(10,21):
    list1.append(i)

list1
#########################################################
out_put=[]
for i in list1:
    if len(i)>3:
        out_put.append(i)
out_put
#########################################################
out=[]
for i in list1:
    if i=='hyd':
        out.append(i)
out
#########################################################################
out=[]
for i in list2:
    if '#' in i:
        out.append(i)

out
```

list-comprehension

In [ ]:
```
**pattern-1**:

#list1=[<output> <forloop>]
```

In [87]:
```
list1=[]
for i in range(10,21):
    list1.append(i)

list1
#################################
#list1=[<output> <forloop>]
list2=[i for i in range(10,21)]
list2
```

Out[87]: [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

In [88]:
```
import random
#num=[]
#for i in range(10):
    #num.append(random.randint(10,100))

num=[random.randint(10,100) for i in range(10)]
num
```

Out[88]:
```
[36, 13, 58, 86, 47, 31, 42, 17, 39, 33]
```

In [ ]:
```
**pattern-2**

# [<output> <forloop> <if condition>]
```

In [89]:
```
list1=['hyd','bengaluru','chennai']
#out_put=[]
#for i in list1:
    #if len(i)>3:
        #out_put.append(i)
#out_put


out=[i for i in list1 if len(i)>3]
out
```

Out[89]:
```
['bengaluru', 'chennai']
```

In [90]:
```
out=[]
for i in list1:
    if i=='hyd':
        out.append(i)
out

list1 = ['benguluru','hyd', 'chennai']
new = [i for i in list1 if i!='hyd']
new
```

Out[90]:
```
['benguluru', 'chennai']
```

In [91]:
```
list1=['hyd','ben#galuru','che#nnai']
out=[i for i in list1 if '#' in i]
out
```

Out[91]:
```
['ben#galuru', 'che#nnai']
```

In [ ]:
```
# There are two list are there
# Qns list =['who is pm of india',
            'How many states are there in india',
            'what is the capital of india']
# Ans list=['Modi','29','Delhi']

# iterate over qns list
# ans=input(i)
# get the ans  you need to chcek ans list
# if it is correct give one mark
# finally you need to display
# out of 3 qns 2 ans are correct and the marks :2
```

In [92]:
```
ans=input("3+5:")
```

```
3+5:8
```

Out[92]:
```
'8'
```

In [ ]:
```
# List comprehenshion
```

In [2]:
```
# only for loop
# yo need to save the output in a list

list1=[]
for i in range(10):
    list1.append(i)
```

In [3]:
```
list1
```

Out[3]:
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [4]:
```
#list2=[<output>  <forloop>]
list2=[i for i in range(10)]
list2
```

Out[4]:
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [8]:
```
# Pattern-2
# for loop   if condition
list1=['hyd','bengaluru','chennai']
# print the elements having len>3
out=[]
for i in list1:
    if len(i)>3:
        out.append(i)

out
```

Out[8]:
```
['bengaluru', 'chennai']
```

In [9]:
```
#out=[<output> <for loop> <if condition>]
out=[i for i in list1 if len(i)>3]
out
```

Out[9]:
```
['bengaluru', 'chennai']
```

In [ ]:
```
list1=[1,2,3]

list2=[]
list2.append(1)
list2.append(2)
list2.append(3)
```

$pattern - 3$:

- for if else

In [10]:
```
# normal code which is having for if else

# wap ask the user print the number is even or odd

for i in range(10):
    if i%2==0:
        print("even")
    else:
        print("odd")
```

```
even
odd
even
odd
even
odd
even
odd
even
odd
```

In [12]:
```
for i in range(10):
    if i%2==0:
        print("even")

# list1=[<output> <for> <if>]
list1=["even" for i in range(10) if i%2==0]
list1
```

```
even
even
even
even
even
```

Out[12]:
```
['even', 'even', 'even', 'even', 'even']
```

In [18]:
```
# for loop:
#     if <condition>:
#         if_out
#     else:
#         else_out        ctrl+/

#[<if_out> if <condition> else <else_out> <for loop>]

# for i in range(10):
#     if i%2==0:
#         print("even")
#     else:
#         print("odd")

[f"{i}:even" if i%2==0 else f"{i}:odd" for i in range(10)]
```

Out[18]:
```
['0:even',
 '1:odd',
 '2:even',
 '3:odd',
 '4:even',
 '5:odd',
 '6:even',
 '7:odd',
 '8:even',
 '9:odd']
```

In [ ]:
```
#pattern-1: [<output> <for loop>]
#pattern-2: [<output> <for loop> <if condition>]
#pattern-3: [<if_out> <if condition> else <else_output> <for loop>]
#dont try to implement elif: there is no elif === else
```

In [19]:
```
dir([])
```

Out[19]:
```
['__add__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'clear',
 'copy',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']
```

clear

In [21]:
```
list1=[1,2,3,4]
list1.clear()
list1
```

Out[21]:
```
[]
```

copy:

In [26]:
```
list1=[1,2,3,4]
list2=list1.copy()
# clear the list1
list1.clear()

print("list1:",list1)
print("list2:",list2)
```

```
list1: []
list2: [1, 2, 3, 4]
```

count

In [27]:
```
list1=[1,2,3,1,1,1]
list1.count(1)
```

Out[27]: 4

In [28]:
```
s='ola ola ola'
s.count('ola')
```

Out[28]: 3

In [32]:
```
s='ola ola ola hai how are you'
list1=s.split()
print(list1)
list1.count('ola')
```

```
['ola', 'ola', 'ola', 'hai', 'how', 'are', 'you']
```

Out[32]: 3

extend

- explore extend
- check the difference of list concatenation
- check the difference of append method
- extend vs append vs concatenation

In [36]:
```
# take two list
list1=[1,2,3]
list2=['A','B','C']
#list1.extend(list2)
list2.extend(list1)
print("list1:",list1)
print("list2:",list2)
```

```
list1: [1, 2, 3]
list2: ['A', 'B', 'C', 1, 2, 3]
```

In [37]:
```
# Concatenation
list1=[1,2,3]
list2=['A','B','C']

print("l1+l2:",list1+list2)  # list1.extend(list2)
print("l2+l1:",list2+list1)  # list2.extend(list1)
print("l1:",list1)
print("l2:",list2)
```

```
l1+l2: [1, 2, 3, 'A', 'B', 'C']
l2+l1: ['A', 'B', 'C', 1, 2, 3]
l1: [1, 2, 3]
l2: ['A', 'B', 'C']
```

In [42]:
```
list1=[1,2,3]
list2=['A','B','C']
list1.append(list2) # [1,2,3,['A','B','C']]
print(list1)
```

```
[1, 2, 3, ['A', 'B', 'C']]
```

In [43]:
```
list1=[1,2,3]
list2=['A','B','C']
list1.extend(list2) # [1,2,3,'A','B','C']
print(list1)
```

```
[1, 2, 3, 'A', 'B', 'C']
```

- list conctenation same as extend method
- list1+list2 is same as list1.extend(list2)
- list2+list1 is same as list2.extend(list1)
- but in concatenation list values will not overwrite
- but in extend method list values will be overwrite

pop-remove

In [46]:
```
# Remove and return item at index (default last).
list1=[1,2,3,'A','B','C']
list1.pop()
list1

# pop will delete the last item by default
# it will also return the element name also
# list will be overwrite
```

Out[46]:
```
[1, 2, 3, 'A', 'B']
```

In [47]:
```
list1=[1,2,3,'A','B','C']
list1.pop(3)
list1
```

Out[47]:
```
[1, 2, 3, 'B', 'C']
```

In [48]:
```
list1=[1,2,3,'A','B','C']
list1.pop(10)
list1
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[48], line 2
      1 list1=[1,2,3,'A','B','C']
----> 2 list1.pop(10)
      3 list1

IndexError: pop index out of range
```

In [49]:
```
list1=[1,2,3,'A','B','C']
list1.remove('A')
list1
```

Out[49]:
```
[1, 2, 3, 'B', 'C']
```

In [52]:
```
list1=[1,2,3,'A','A','A','B','C']
list1.remove('A')
# Remove will take value as a argument
# It is not return anyting
# It will remove only first occurence
# If element will not present raise a value error
# It will overwrite
list1
```

Out[52]:
```
[1, 2, 3, 'A', 'A', 'B', 'C']
```

In [ ]:
```
# pop will delete the last item by default
# it will also return the element name also
# list will be overwrite
# if index is not present, it raise a index error

# Remove will take value as a argument
# It is not return anyting
# It will remove only first occurence
# If element will not present raise a value error
# It will overwrite
```

In [59]:
```
list1=[1,2,3,100,100,100,100,200,200,300]
# I want to remove the first 100
# index of first 100 is 3
# you should not count index manually
# you can use index method
i1=list1.index(100)
i2=list1.index(100,i1+1)
i1,i2
list1.pop(i2)

list1.pop(list1.index(100,list1.index(100)+1))
```

Out[59]:
```
(3, 4)
```

In [63]:
```
s='hai hai hai'
i1=s.index('a')
i2=s.index('a',i1+1)
i3=s.index('a',i2+1)
i1,i2,i3
```

Out[63]:
```
(1, 5, 9)
```

insert

- check the difference with append
- insert vs append

In [64]:
```
list1=['Apple','Banana']
# you want add cherry  at last
# you want add cherry between apple and banana
# what is the cherry index=1
list1.insert(1,'cherry')
list1
```

Out[64]:
```
['Apple', 'cherry', 'Banana']
```

In [66]:
```
list1=['Apple','Banana']
# you want add cherry  at last
list1.append('cherry')
list1
```

Out[66]:
```
['Apple', 'Banana', 'cherry']
```

In [69]:
```
list1=['Apple','Banana']
i1=list1.index(list1[-1])  # list1[-1]='Banana'
#i1=1
#i1+1=2
list1.insert(i1+1,'cherry')
list1
```

Out[69]:
```
['Apple', 'Banana', 'cherry']
```

In [70]:
```
list1=['Apple','Banana']
list1.insert(-1,'cherry')
list1
```

Out[70]:
```
['Apple', 'cherry', 'Banana']
```

In [71]:
```
list1=['Apple','Banana']
list1.insert(len(list1),'Cherry')
list1
```

Out[71]:
```
['Apple', 'Banana', 'Cherry']
```

In [72]:
```
dir([])
```

Out[72]:
```
['__add__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'clear',
 'copy',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']
```

revesre

In [73]:
```
list1=[1,2,3,'A','B','C']
list1.reverse()
list1
```

Out[73]:
```
['C', 'B', 'A', 3, 2, 1]
```

In [ ]:
```
key_Word: reversed

reverse method  and reversed keyword
```

In [77]:
```
list1=[1,2,3,'A','B','C']
list2=[]
for i in reversed(list1):
    list2.append(i)
list2
```

Out[77]:
```
['C', 'B', 'A', 3, 2, 1]
```

In [ ]:
```
reverse is a method : list

reverse is not avaialble : strings

reversed  method is applicable for everyone

<list1>.reverse()

reversed(list1)
```

In [83]:
```
str1='python'
str2=''
for i in reversed(str1):
    str2=str2+i
str2


list1=[1,2,3,'A','B','C']
list2=[]
for i in reversed(list1):
    list2.append(i)
list2
```

Out[83]:
```
'nohtyp'
```

In [ ]:
```
sort is a method

sorted  is a keyword
```

- append extend insert
- pop remove
- pop index
- clear copy count
- reverse reversed
- sort sorted

In [ ]:
```
# string assignment
```

In [ ]: